

```

#version 430 core

const int N = 4;
const int SIZE = 2;
const int CUBIES_COUNT = SIZE * SIZE * SIZE * SIZE;
const int XFORM_SIZE = N * N;
const int GENES_COUNT = 32;
const int POPULATION_COUNT = 1024;
const int PLANES_COUNT = N * (N - 1) / 2;
const float C = (SIZE - 1) / 2;
//const int CUBIES_SIZE = (XFORM_SIZE + N) * CUBIES_COUNT;

layout(local_size_x = 1) in;

struct XForm
{
    float[XFORM_SIZE] M;
    float[N] Origin;
};

layout(std430, binding = 0) buffer Cubies {
    XForm[CUBIES_COUNT] cubies;
};

layout(std430, binding = 1) buffer SolvedCubies {
    int solvedCubies[];
};

layout(std430, binding = 2) buffer ActiveCubies {
    int activeCubies[];
};

layout(std430, binding = 4) buffer planes
{
    vec2 Planes[PLANES_COUNT];
};

struct Chromosome
{
    float Genes[GENES_COUNT];
    float MoveCount;
    float Fitness;
};

layout(std430, binding = 5) buffer Population
{
    Chromosome population[POPULATION_COUNT];
};

int getAngle(float cosA, float sinA)
{
    if (cosA > 0.5) return 0;
    if (sinA > 0.5) return 1;
    if (cosA < -0.5) return 2;
    if (sinA < -0.5) return 3;
    return 0;
}

```

```

vec2 setAngle(int angle)
{
    float cosA = 0;
    float sinA = 0;
    switch (angle)
    {
        case 0: cosA = 1; break;
        case 1: sinA = 1; break;
        case 2: cosA = -1; break;
        case 3: sinA = -1; break;
    }
    return vec2(cosA, sinA);
}

void RotateVec(inout float[N] v, int axis1, int axis2, vec2 rot)
{
    float a = v[axis1];
    float b = v[axis2];
    v[axis1] = rot.x * a - rot.y * b;
    v[axis2] = rot.y * a + rot.x * b;
}

void RotateMat(inout float[XFORM_SIZE] mat, int axis1, int axis2, vec2
rot)
{
    for (int i = axis1, j = axis2; i < XFORM_SIZE; i += N, j += N)
    {
        float a = mat[i];
        float b = mat[j];
        mat[i] = rot.x * a - rot.y * b;
        mat[j] = rot.y * a + rot.x * b;
    }
}

vec2[PLANES_COUNT] getEulerAngles(float[XFORM_SIZE] A)
{
    vec2[PLANES_COUNT] angles;
    for (int i = 0; i < Planes.length(); i++)
    {
        int axis1 = int(Planes[i].x);
        int axis2 = int(Planes[i].y);
        float a = A[axis1 + axis1 * N];
        float b = A[axis2 + axis1 * N];
        float r = sqrt(a * a + b * b);
        if (r < 0.1)
            angles[i] = vec2(1, 0);
        else
        {
            float cosA = a / r;
            float sinA = b / r;
            RotateMat(A, axis1, axis2, vec2(cosA, -sinA));
            angles[i] = vec2(cosA, sinA);
        }
    }
    //float[N] scale;
    //for (int i = 0; i < N; i++)
    //    scale[i] = (float)A.Cols[0].Norm;
    //var error = (A - TAffine.CreateScale(scale).M).Norm;

```

```

        //if (error > 1E-3)
        //    ;
        return angles;
    }

int getState(XForm transform)
{
    vec2[PLANES_COUNT] EulerAngles = getEulerAngles(transform.M);
    int state = 0;
    int shift = (PLANES_COUNT - 1) << 1;
    for (int i = 0; i < PLANES_COUNT; i++)
    {
        int angle = getAngle(EulerAngles[i].x, EulerAngles[i].y);
        state |= angle << shift;
        shift -= 2;
    }
    return state;
}

int getRotationCount(int state)
{
    int RotationCount = 0;
    for (int i = 0; i < PLANES_COUNT; i++)
    {
        if ((state & 3) > 0)
            RotationCount++;
        state >>= 2;
    }
    return RotationCount;
}

struct Move
{
    int Axis;
    int Slice;
    int Plane;
    int Angle;
};

Move getMove(int code)
{
    int size = SIZE * PLANES_COUNT * 3;
    int axis = code / size;
    code -= axis * size;
    size /= SIZE;
    int slice = code / size;
    code -= slice * size;
    size /= PLANES_COUNT;
    int plane = code / size;
    code -= plane * size;
    int angle = code;
    return Move(axis, slice, plane, angle);
}

float evaluateCubies(XForm[CUBIES_COUNT] workCubies)
{
    float score = 0;
    int scrambled = 0;

```

```

float maxClusterState = float(1 << 2 * PLANES_COUNT) *
activeCubies.length() * N;
for (int i = 0; i < activeCubies.length(); i++)
{
    //XForm cubie;
    //cubie = workCubies[activeCubies[i]];
    //score += cubie.M[0] + cubie.M[1] + cubie.M[2] + cubie.M[3];
    int state = getState(workCubies[activeCubies[i]]);
    if (state != 0)
    {
        score += maxClusterState + state + (getRotationCount(state) <<
PLANES_COUNT);
        scrambled++;
    }
}
score /= maxClusterState * (activeCubies.length() + 1);
if (scrambled == 1)
    score *= 2;
for (int i = 0; i < solvedCubies.length(); i++)
{
    int state = getState(workCubies[solvedCubies[i]]);
    if (state != 0)
        score++;
}
return 100 * score;
}

```

```

void Rotate(inout XForm cubie, int plane, int angle)
{
    int axis1 = int(Planes[plane].x);
    int axis2 = int(Planes[plane].y);
    vec2 rot = setAngle(angle);
    RotateMat(cubie.M, axis1, axis2, rot);
    RotateVec(cubie.Origin, axis1, axis2, rot);
}

```

```

void Turn(inout XForm[CUBIES_COUNT] workCubies, Move move)
{
    for (int i = 0; i < workCubies.length(); i++)
    {
        float v = round(workCubies[i].Origin[move.Axis] + C);
        if (int(v) == move.Slice)
            Rotate(workCubies[i], move.Plane, move.Angle + 1);
    }
}

```

```

void main()
{
    int specimenID = int(gl_GlobalInvocationID.x);
    Chromosome specimen = population[specimenID];
    float bestFitness = 100 * cubies.length();
    XForm[CUBIES_COUNT] workCubies = cubies;
    for (int i = 0; i < GENES_COUNT; i++)
    {
        Turn(workCubies, getMove(int(specimen.Genes[i])));
        float fitness = evaluateCubies(workCubies);
        if (fitness < bestFitness)

```

```
        {
            bestFitness = fitness;
            specimen.MoveCount = i + 1;
        }
    specimen.Fitness = bestFitness;
    population[specimenID] = specimen;
}
```